



CS240 Algorithm Design and Analysis

Lecture 2

Greedy Algorithms

Qingying Hao

Spring 2026

2026.03.05



A greedy algorithm always makes the choice that looks best at the moment and adds it to the current partial solution

Greedy algorithms don't always yield optimal solutions, but when they do, they're usually the simplest and most efficient algorithms available

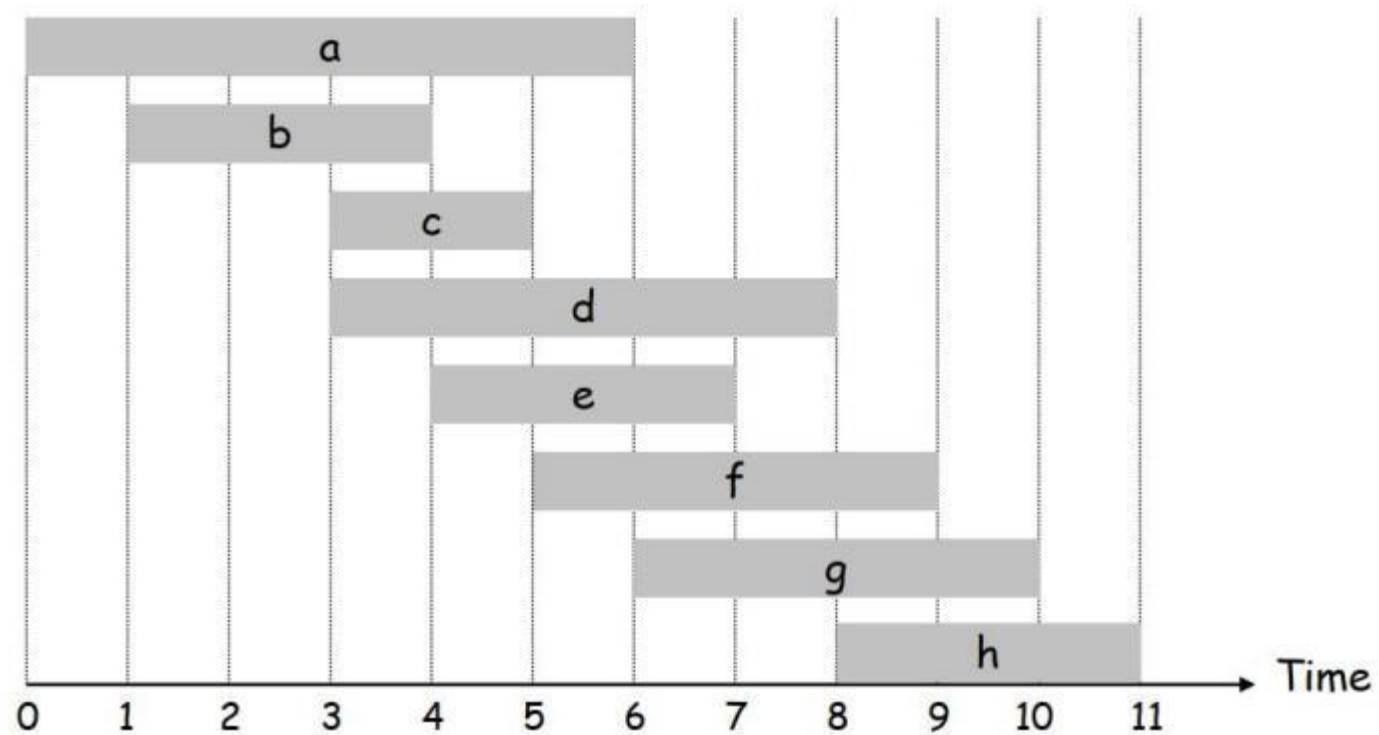




Interval Scheduling



- Interval scheduling
 - Job j starts at s_j and finishes at f_j
 - Two jobs are **compatible** if they don't overlap
 - Goal: find maximum size subset of mutually compatible jobs

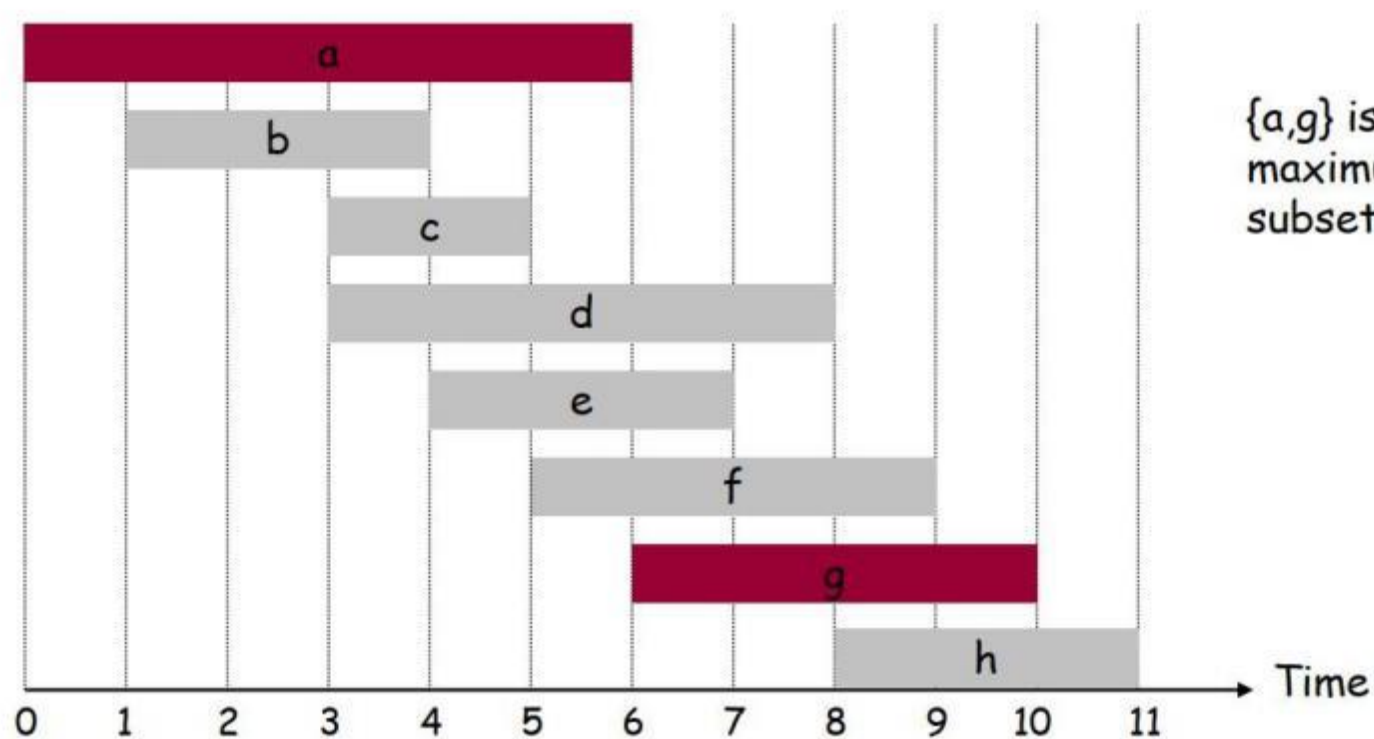




Interval Scheduling



- Interval scheduling
 - Job j starts at s_j and finishes at f_j
 - Two jobs are **compatible** if they don't overlap
 - Goal: find maximum size subset of mutually compatible jobs



{a,g} is NOT a maximum-size subset.

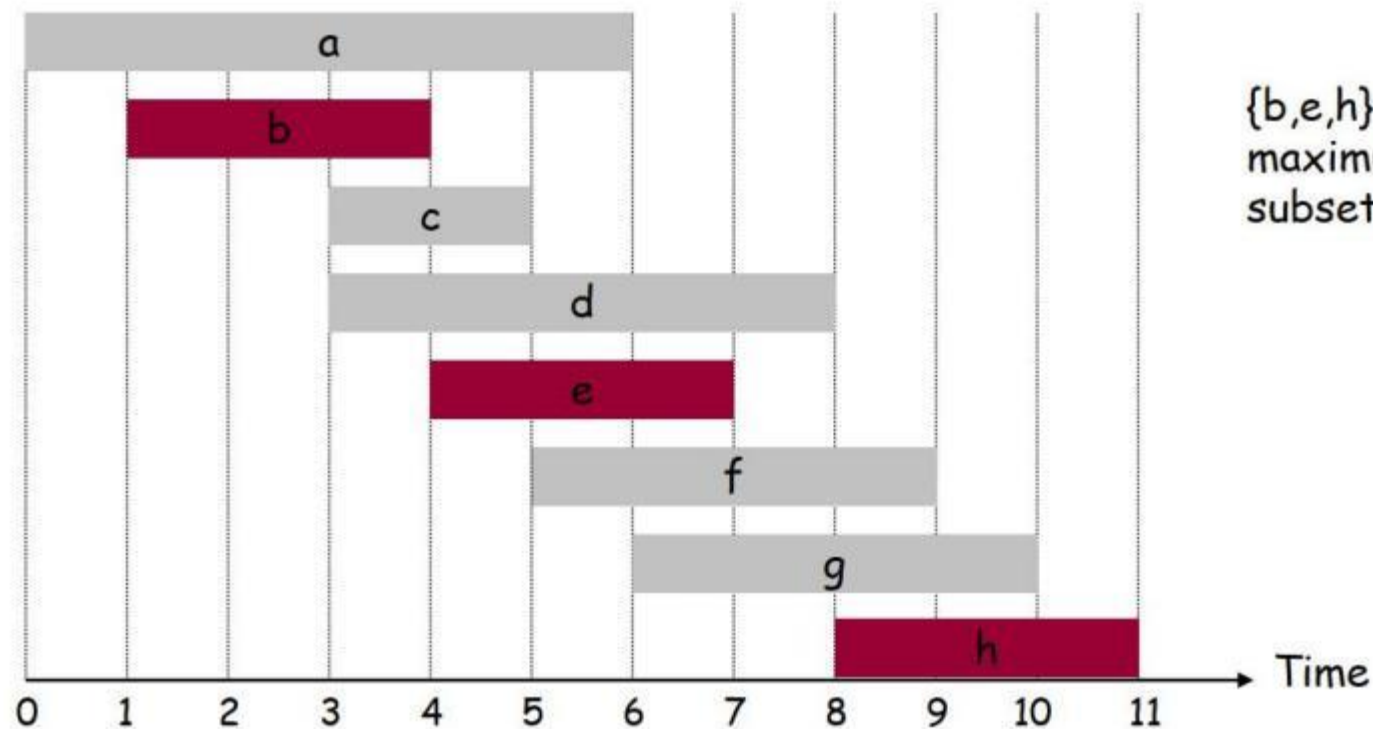




Interval Scheduling



- Interval scheduling
 - Job j starts at s_j and finishes at f_j
 - Two jobs are **compatible** if they don't overlap
 - Goal: find maximum size subset of mutually compatible jobs



$\{b, e, h\}$ is a maximum-size subset.





Interval Scheduling: Greedy Algorithms

- Greedy template. Consider jobs in some order. Take each job provided it's compatible with the ones already taken
- Usually, many compatible jobs can co-exist
- Need to choose a rule specifying which job to choose next
- Three possible rules are:
 - [Earliest start time]
 - Consider jobs in ascending order of start time s_j
 - [Earliest finish time]
 - Consider jobs in ascending order of finish time f_j
 - [Shortest interval]
 - Consider jobs in ascending order of interval length $f_j - s_j$
 - [Fewest conflicts]
 - For each job, count the number of conflicting jobs c_j . Schedule in ascending order of conflicts c_j





Interval Scheduling: Greedy Algorithms



- [Fewest conflicts]
- For each job, count the number of conflicting jobs c_j . Schedule in ascending order of conflicts c_j

job	interval
A	1–4
B	2–5
C	3–6
D	6–8

A: overlap: B, C

$$c_A = 2$$

B: overlap: A, C

$$c_B = 2$$

C: overlap: A, B

$$c_C = 2$$

D: overlap: none

$$c_D = 0$$

Greedy: Select D





Interval Scheduling: Greedy Algorithms

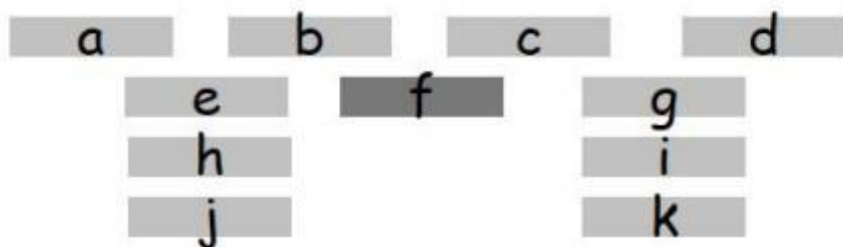
- Greedy template. Consider jobs in some order. Take a job provided it's compatible with the ones already taken



breaks earliest start time



breaks shortest interval



breaks fewest conflicts

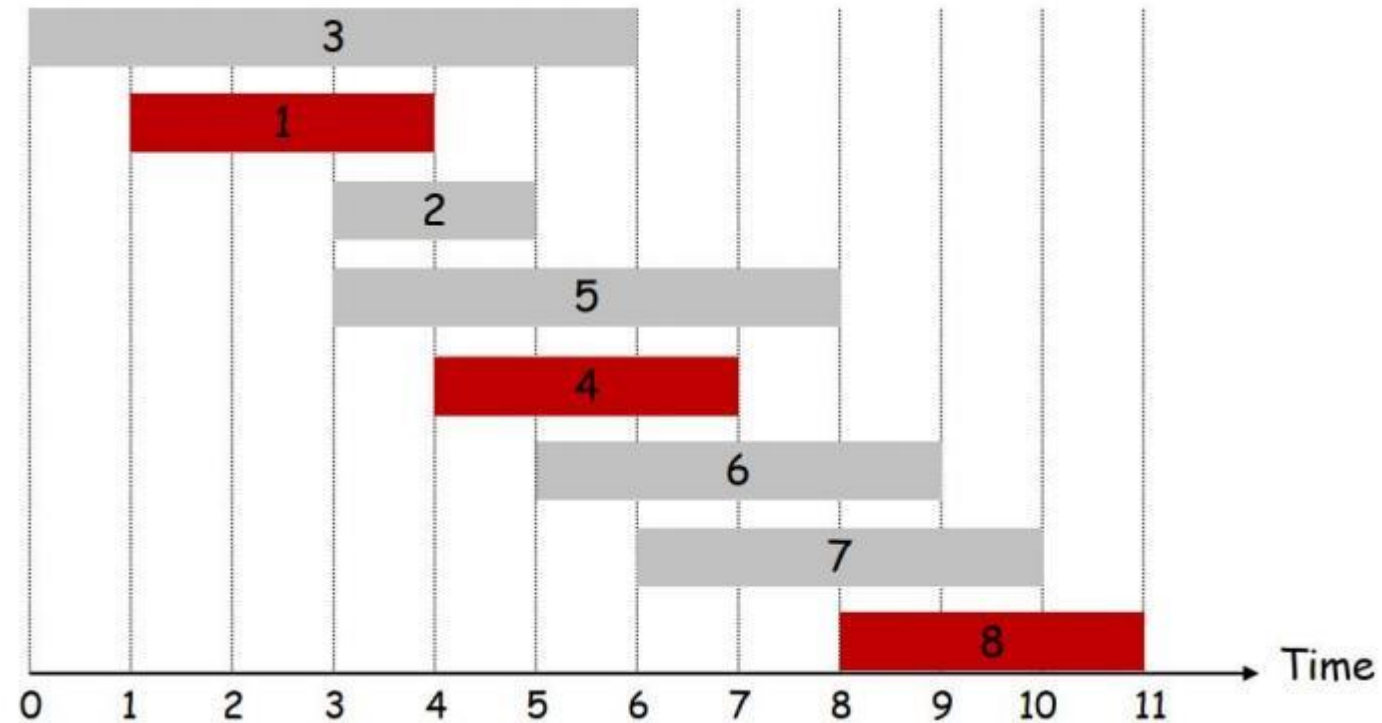
Examples above provide counterexamples for the three rules proposed.
For each of them, provides an example input for which the following that rule yields a non-optimal (i.e., non max-size) schedule





Interval Scheduling: Greedy Algorithm

- Greedy algorithm. Consider jobs in increasing order of **finish time**. Take each job provided it's compatible with the ones already taken





Interval Scheduling: Greedy Algorithm

- Greedy algorithm. Consider jobs in increasing order of **finish time**. Take each job provided it's compatible with the ones already taken

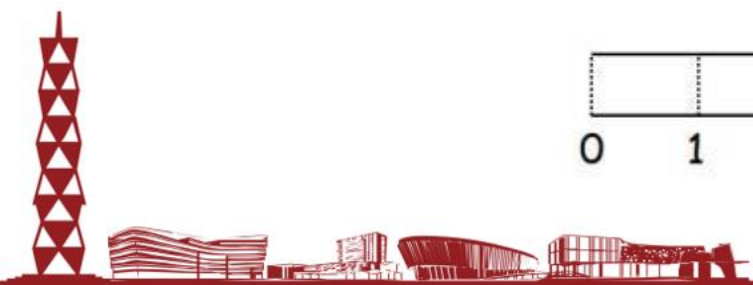
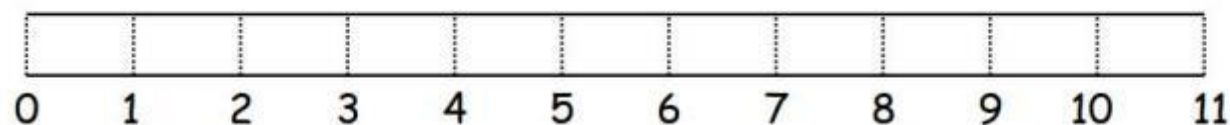
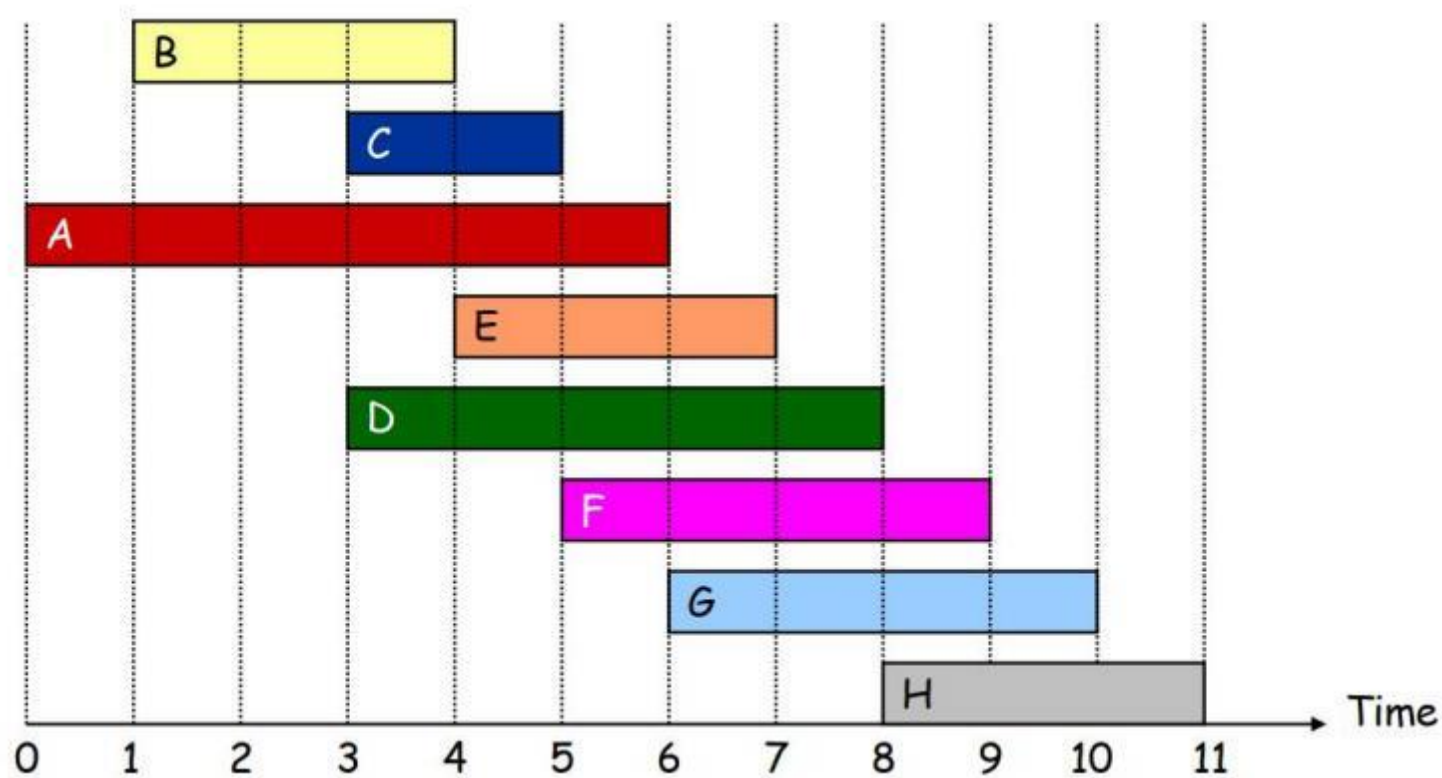
```
Sort jobs by finish times so that  $f_1 \leq f_2 \leq \dots \leq f_n$   
 $A \leftarrow \emptyset, \text{ last} \leftarrow 0$   
for  $j \leftarrow 1$  to  $n$   
    if  $s_j \geq \text{last}$  then  $A \leftarrow A \cup \{j\}, \text{ last} \leftarrow f_j$   
return  $A$ 
```

- Running time dominated by **cost of sorting: $(n \log n)$**
- Remember **the finish time** of the last job (j^*) added to A
- Job j is compatible with A if $s_j \geq \text{last}$ (i.e., f_{j^*})
- **Remember:** Correctness (optimality) of greedy algorithms is usually not obvious.
- **Need to prove!**



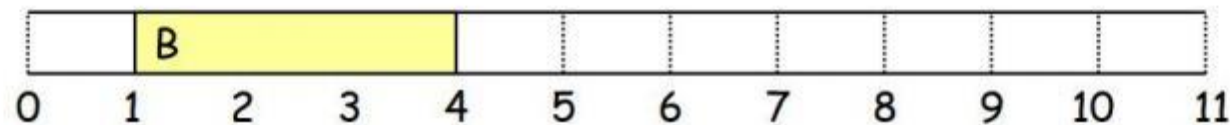
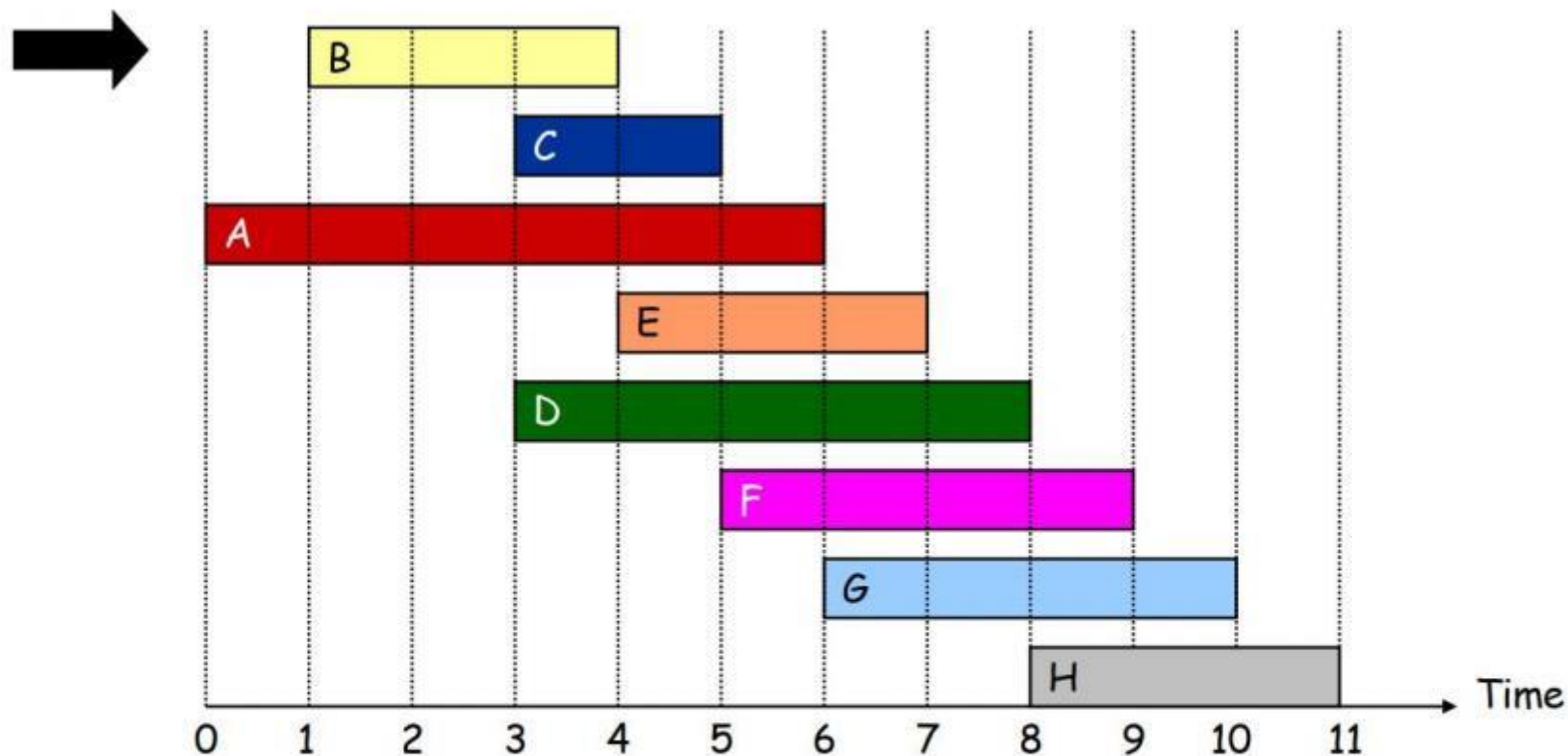


Interval Scheduling



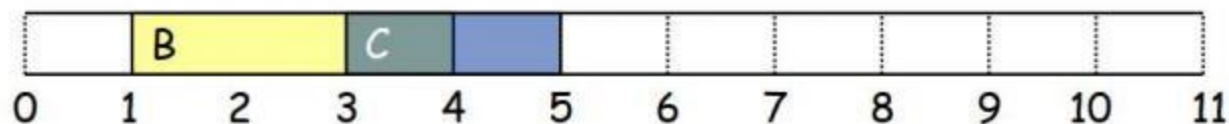
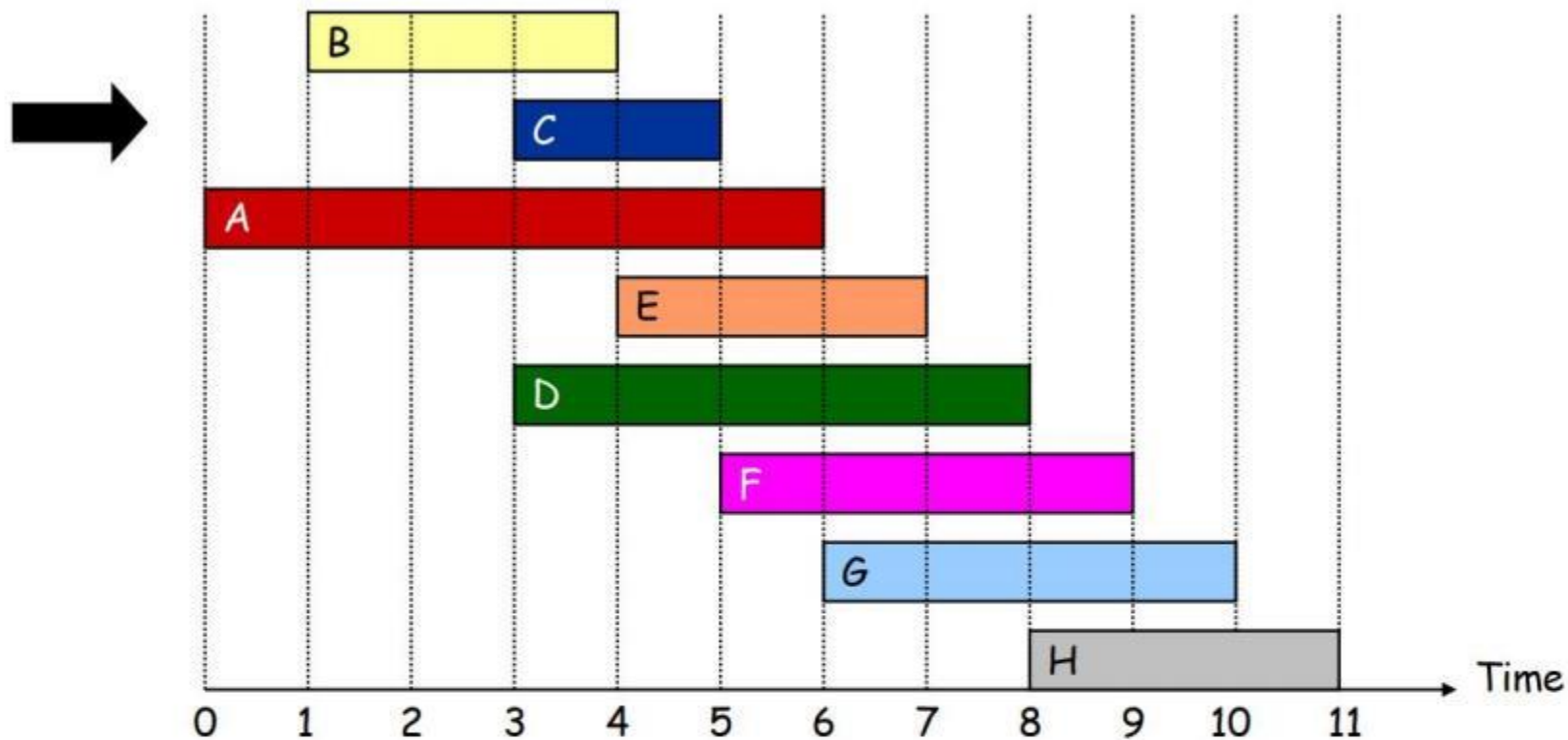


Interval Scheduling



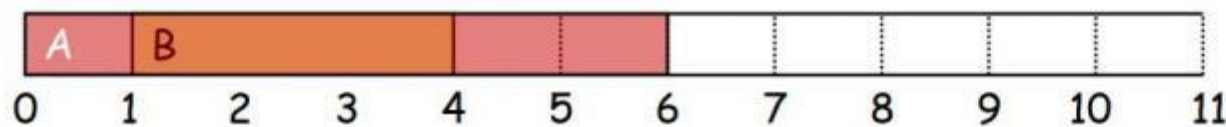
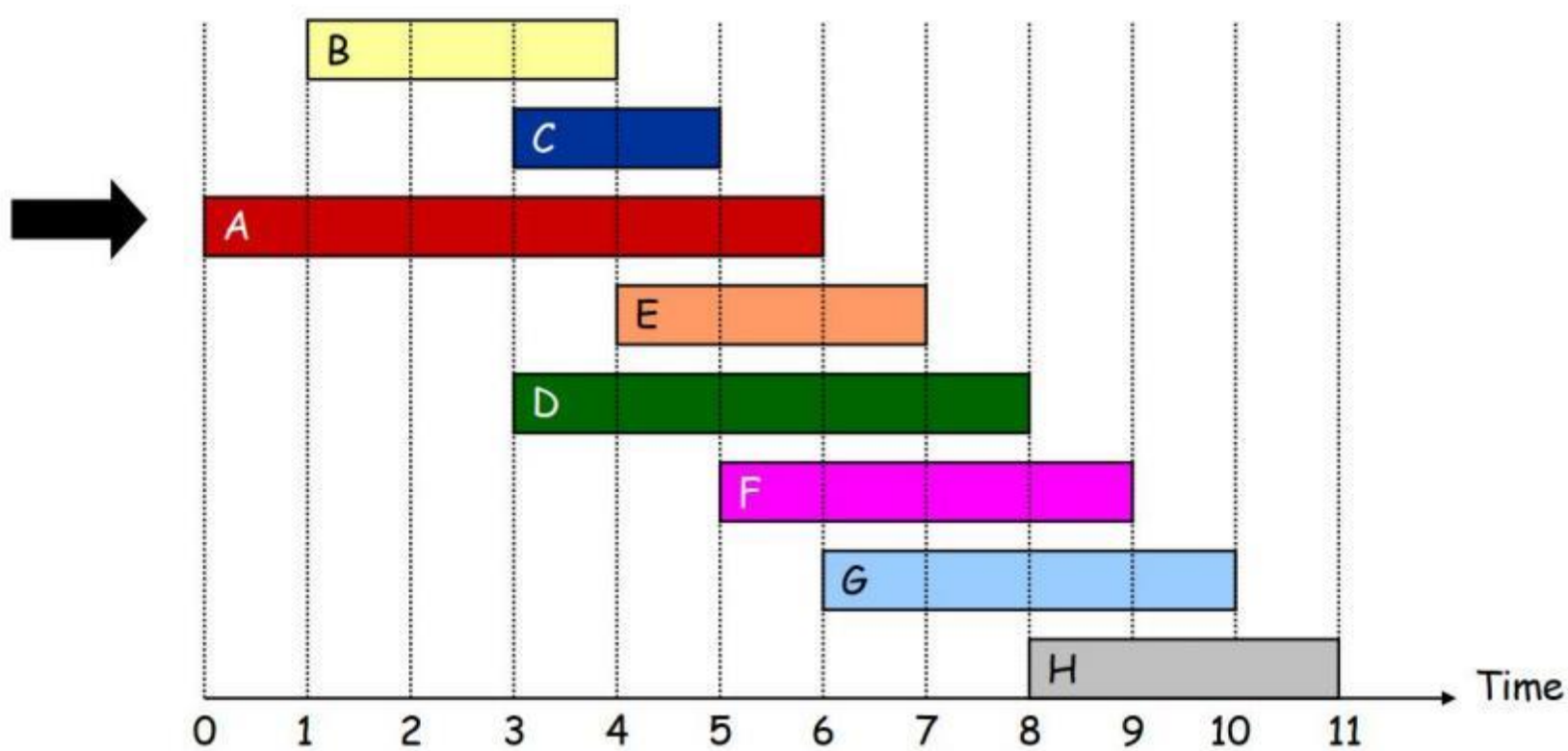


Interval Scheduling



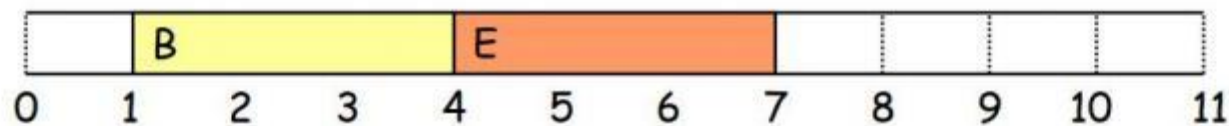
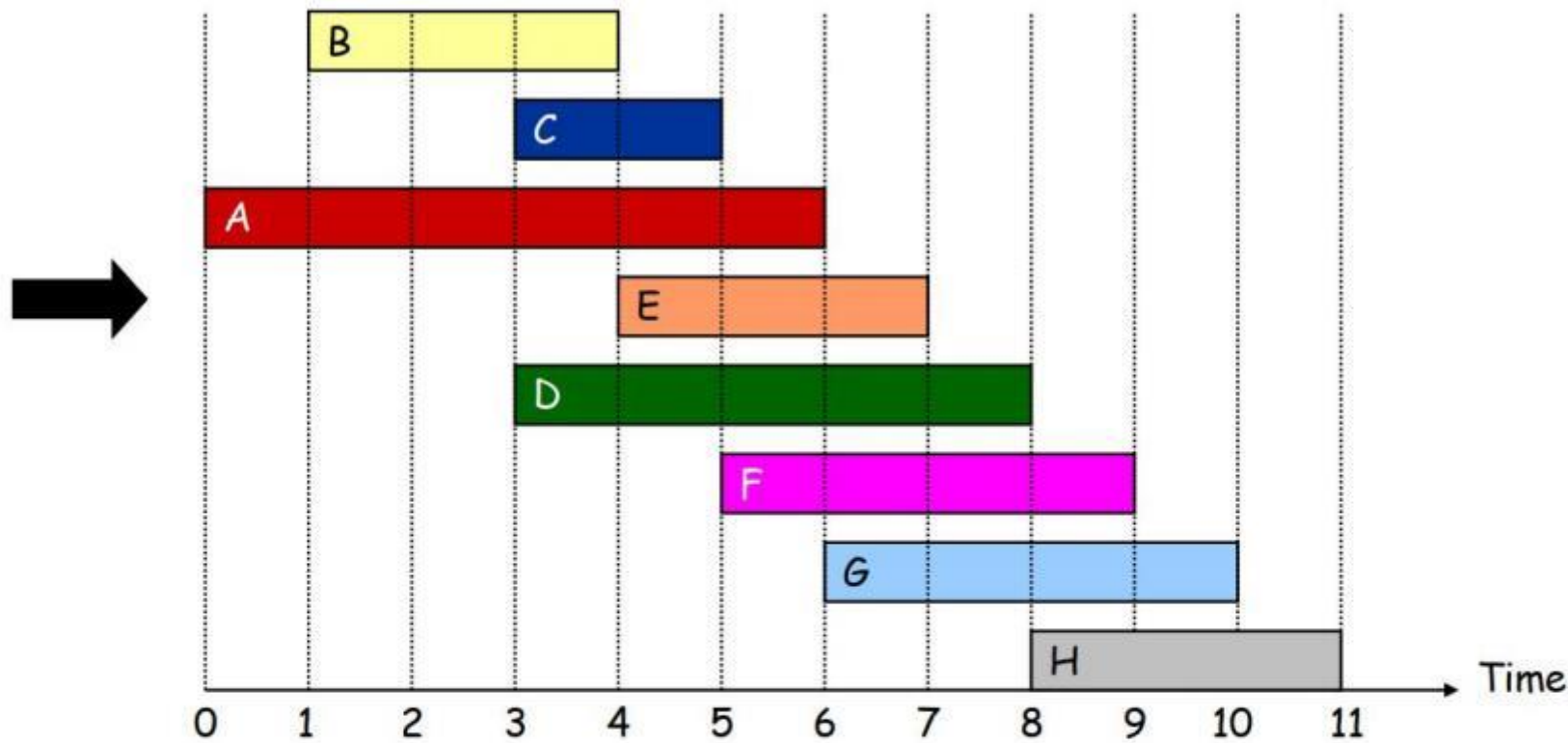


Interval Scheduling



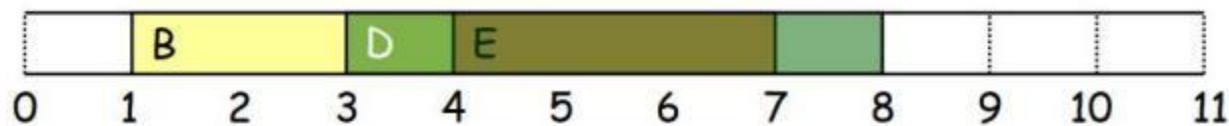
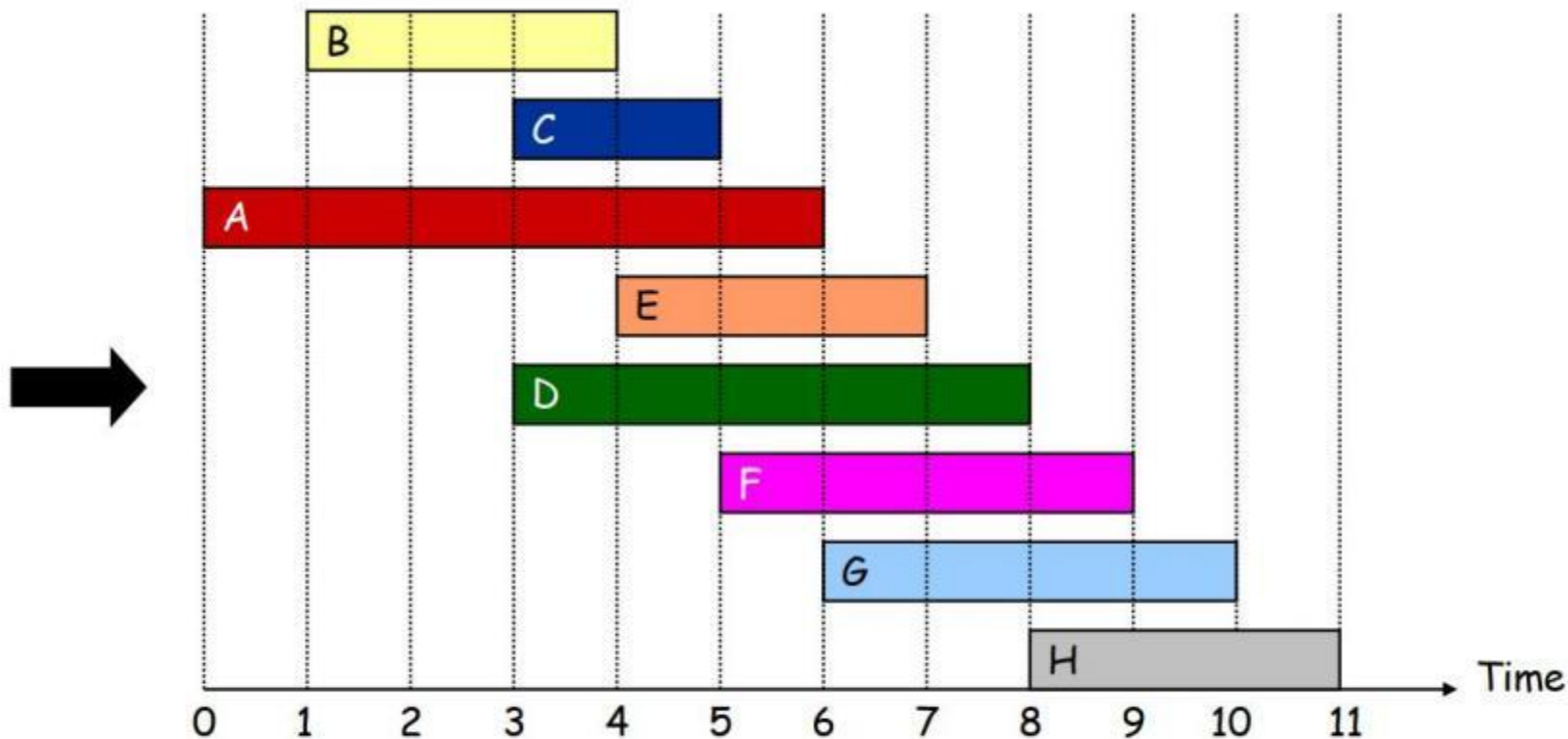


Interval Scheduling



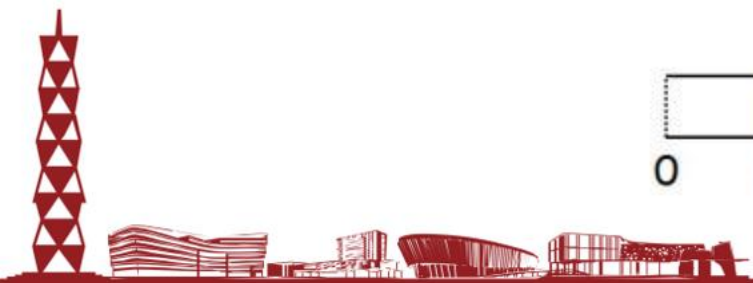
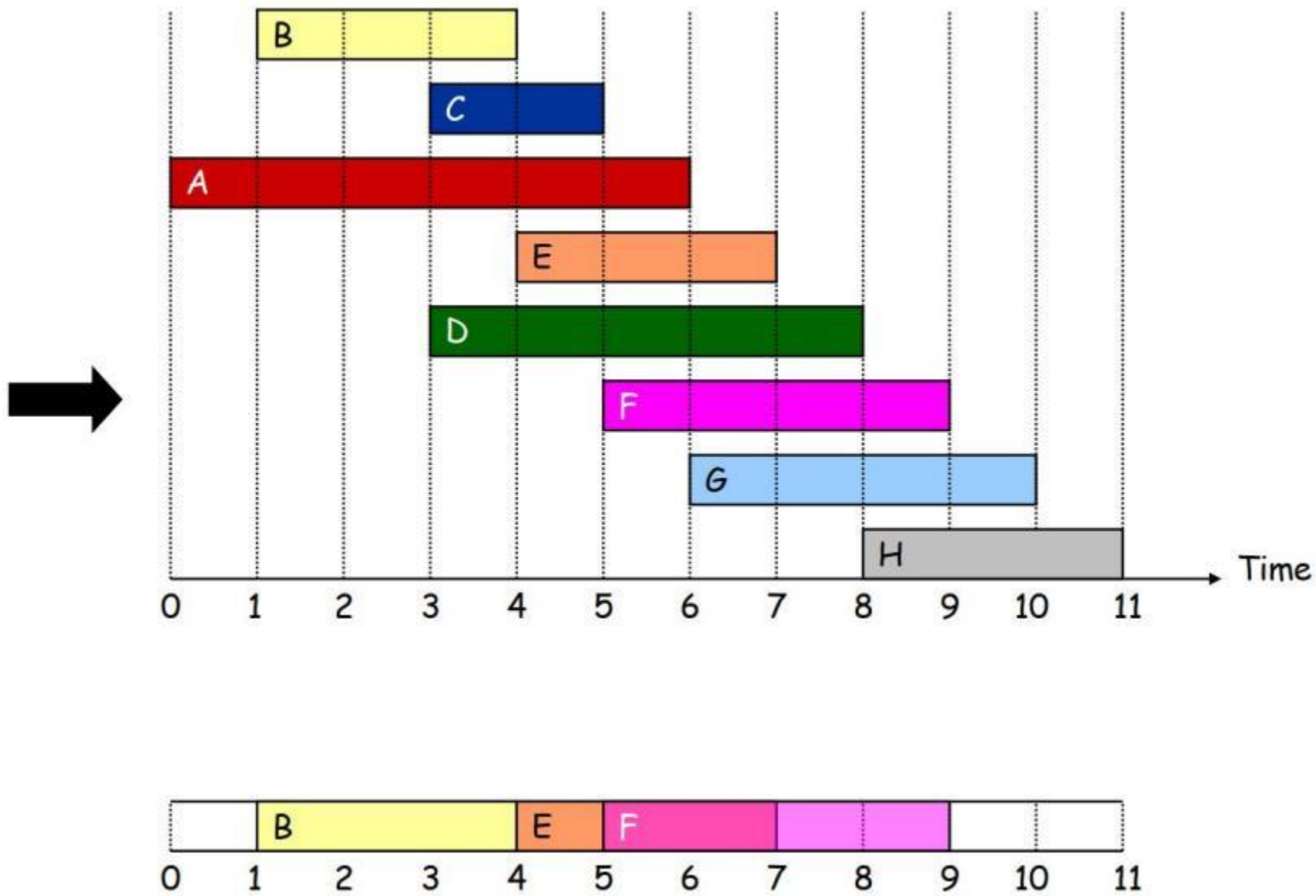


Interval Scheduling



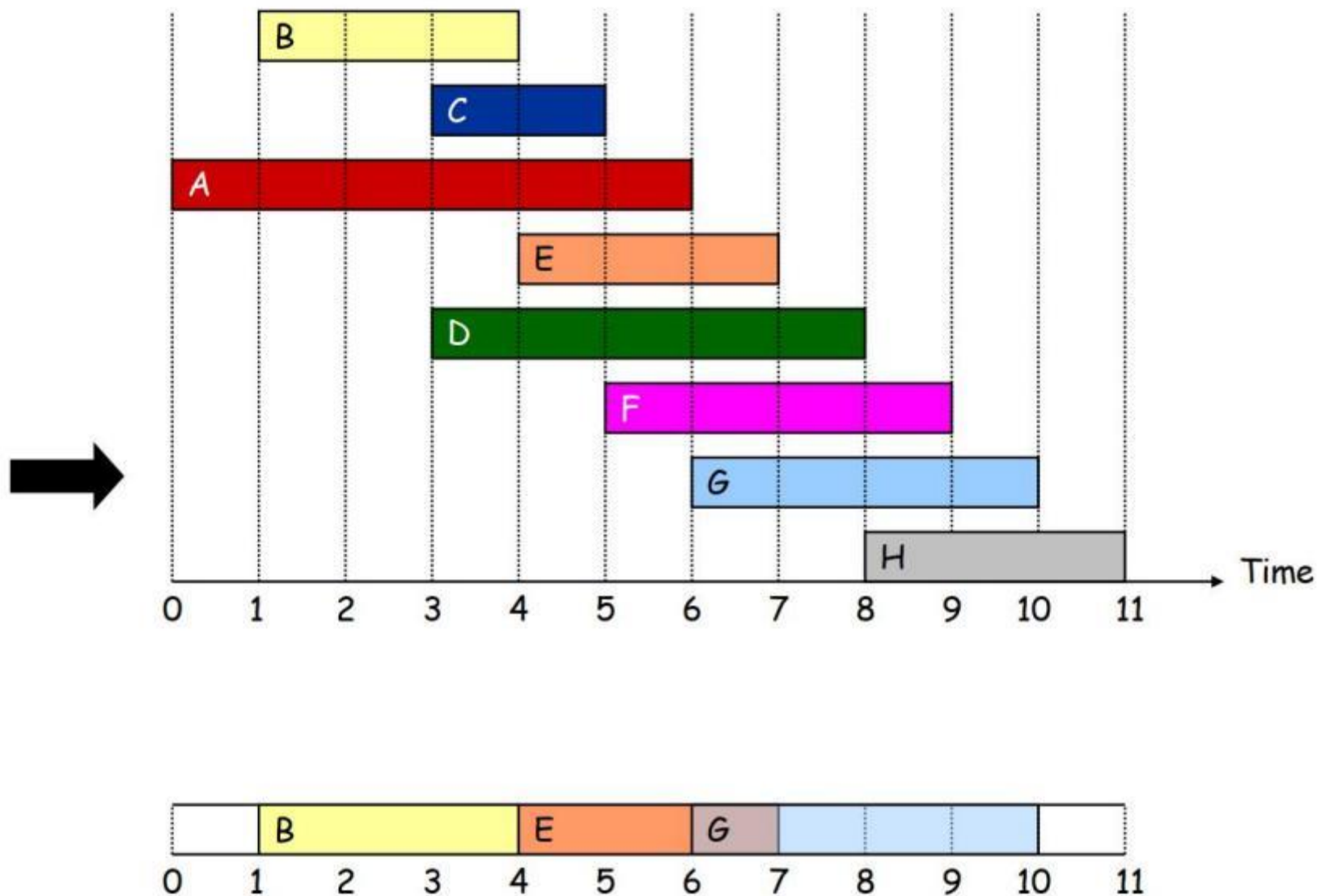


Interval Scheduling



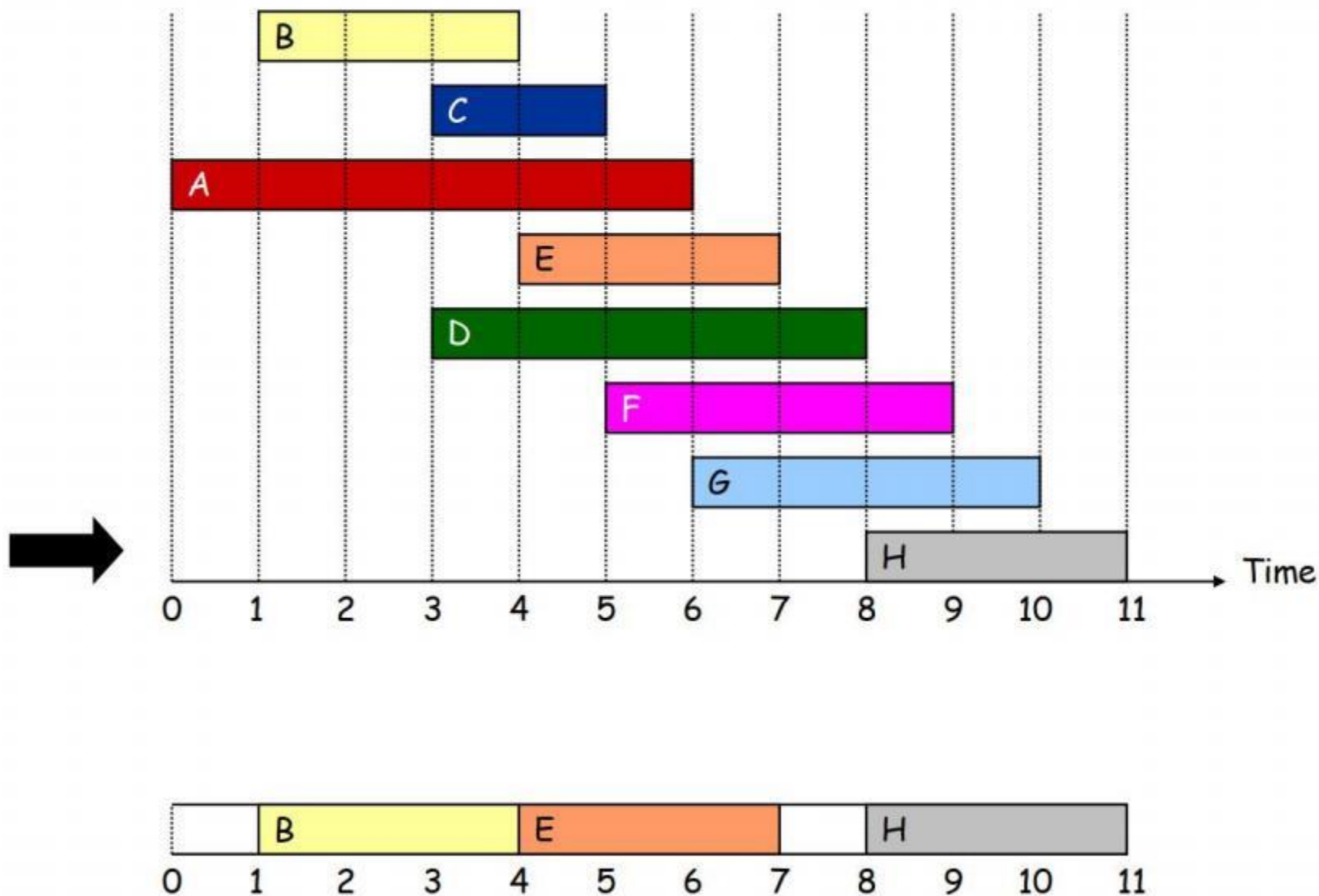


Interval Scheduling





Interval Scheduling



Interval Scheduling: Correctness

- Theorem. Greedy algorithm is optimal.
- Proof (**by contradiction**)
 - Assume Greedy is different from OPT (Optimal solution). Let's see what's different.
 - Let $i_1, i_2, \dots, i_k$ denote the set of jobs selected by greedy.
 - Let $j_1, j_2, \dots, j_m$ denote set of jobs in the optimal solution.
 - Find largest possible value of r such that $i_1 = j_1, i_2 = j_2, \dots, i_r = j_r$

By definition of Greedy, job i_{r+1} finishes before j_{r+1}

Greedy:



OPT:

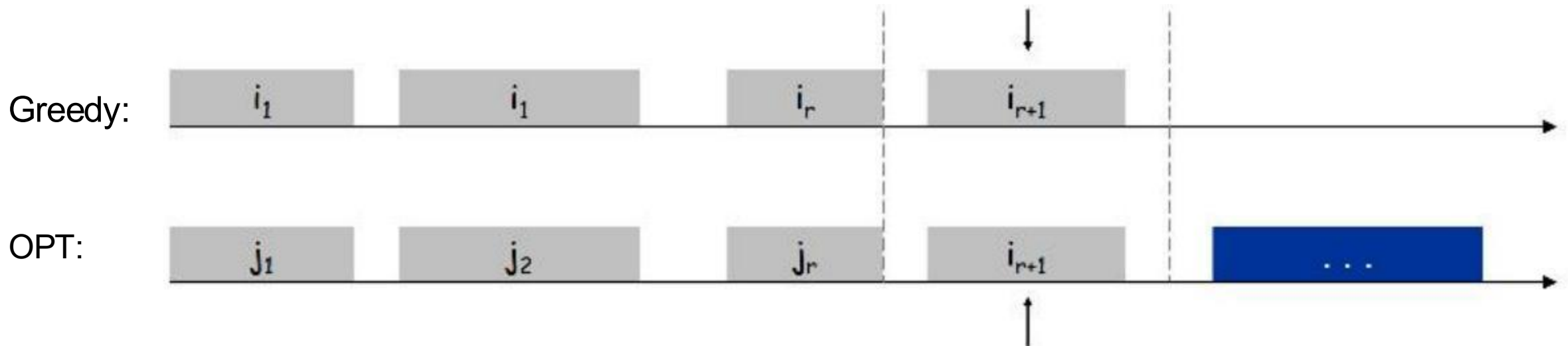


Why not replace job j_{r+1} with job i_{r+1}

Interval Scheduling: Analysis

- Theorem. Greedy algorithm is optimal.
- Pf. (by contradiction)
 - Assume greedy is not optimal, and let's see what happens
 - Let $i_1, i_2, \dots, i_k$ denote the set of jobs selected by greedy.
 - Let $j_1, j_2, \dots, j_k$ denote set of jobs in the optimal solution with $i_1 = j_1, i_2 = j_2, \dots, i_r = j_r$
for the largest possible value of r .

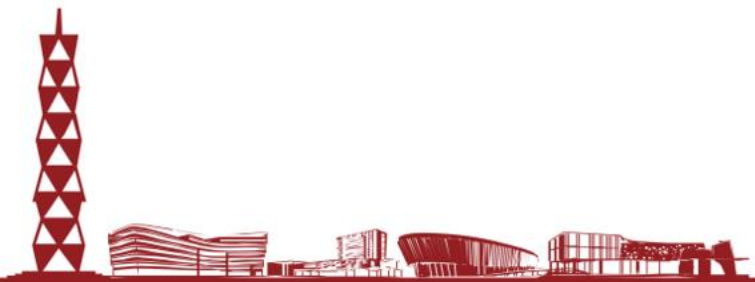
By definition of Greedy, job i_{r+1} finishes before j_{r+1}



Solution still feasible and optimal, but contradicts maximality of r



Scheduling to Minimize Lateness

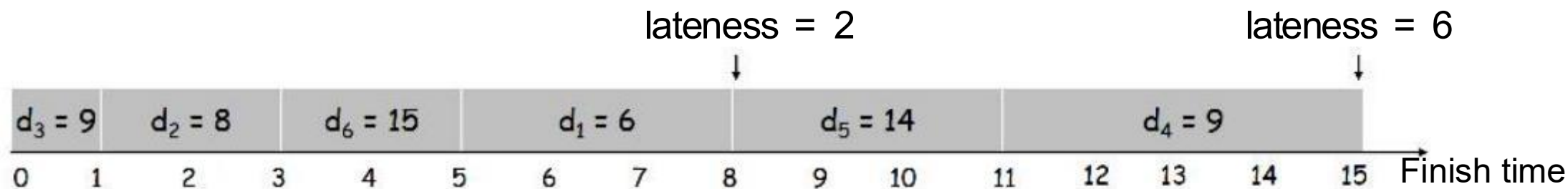


Scheduling to Minimizing Lateness

• Minimizing lateness problem

- Single resource processes one job at a time
- Job j requires t_j units of processing time and is due at time d_j
- If j starts at time s_j , it finishes at time $f_j = s_j + t_j$
- Lateness: $l_j = \max\{0, f_j - d_j\}$
- Goal: schedule all jobs to minimize **maximum** lateness $L = \max l_j$

	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15





Minimizing lateness: Greedy Algorithms

- Greedy template: Consider jobs in some order
- [Shortest processing time first] Consider jobs in ascending order of processing time t_j
- [Earliest deadline first] Consider jobs in ascending order of deadline d_j
- [Smallest slack] Consider jobs in ascending order of slack $d_j - t_j$





Minimizing lateness: Greedy Algorithms

- Greedy template: Consider jobs in some order
- [Shortest processing time first] Consider jobs in ascending order of processing time t_j

	1	2
t_j	1	10
d_j	100	10

counterexample

1->2

start = 0

finish = 1.

lateness = $\max(0, 1-100)=0$

- [Earliest deadline first] Consider jobs in ascending order of deadline d_j
- [Smallest slack] Consider jobs in ascending order of slack $d_j - t_j$

start = 1

finish = 11

lateness = $\max(0, 11-10)=1$

	1	2
t_j	1	10
d_j	2	10

counterexample





Minimizing lateness: Greedy Algorithms

- Greedy template: Consider jobs in some order
- [Shortest processing time first] Consider jobs in ascending order of processing time t_j

	1	2
t_j	1	10
d_j	100	10

counterexample

2→1

start = 0

finish = 10

lateness = $\max(0, 10 - 10) = 0$

- [Earliest deadline first] Consider jobs in ascending order of deadline d_j
- [Smallest slack] Consider jobs in ascending order of slack $d_j - t_j$

start = 10

finish = 11

lateness = $\max(0, 11 - 100) = 0$

	1	2
t_j	1	10
d_j	2	10

counterexample



Minimizing lateness: Greedy Algorithms

- Greedy template: Consider jobs in some order
- [Shortest processing time first] Consider jobs in ascending order of processing time t_j

	1	2
t_j	1	10
d_j	100	10

counterexample

- [Earliest deadline first] Consider jobs in ascending order of deadline d_j

- [Smallest slack] Consider jobs in ascending order of slack $d_j - t_j$

	1	2
t_j	1	10
d_j	2	10

counterexample

2→1

start = 0

finish = 10

lateness: $\max(0, 10 - 10) = 0$

start = 10

finish = 11

lateness: $\max(0, 11 - 2) = 9$



Minimizing lateness: Greedy Algorithms

- Greedy template: Consider jobs in some order
- [Shortest processing time first] Consider jobs in ascending order of processing time t_j

	1	2
t_j	1	10
d_j	100	10

counterexample

- [Earliest deadline first] Consider jobs in ascending order of deadline d_j
- [Smallest slack] Consider jobs in ascending order of slack $d_j - t_j$

	1	2
t_j	1	10
d_j	2	10

counterexample

1→2

Start = 0, finish = 1

$$lateness = \max(0, 1 - 2) = 0$$

Start = 1, finish = 11

$$lateness = \max(0, 11 - 10) = 1$$





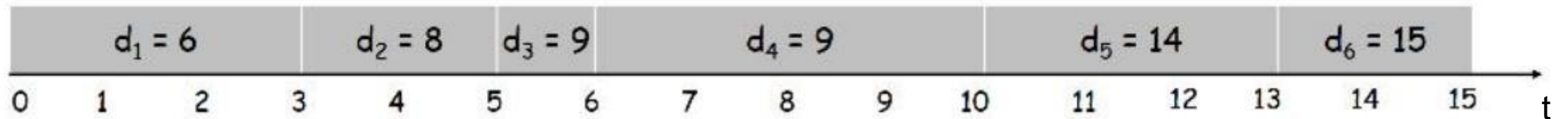
Minimizing Lateness: Greedy Algorithm

- Greedy algorithm. Earliest deadline first

```
Sort n jobs by deadline so that  $d_1 \leq d_2 \leq \dots \leq d_n$   
  
 $t \leftarrow 0$   
for  $j = 1$  to  $n$   
    Assign job  $j$  to interval  $[t, t + t_j]$   
     $s_j \leftarrow t, f_j \leftarrow t + t_j$   
     $t \leftarrow t + t_j$   
output intervals  $[s_j, f_j]$ 
```

	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15

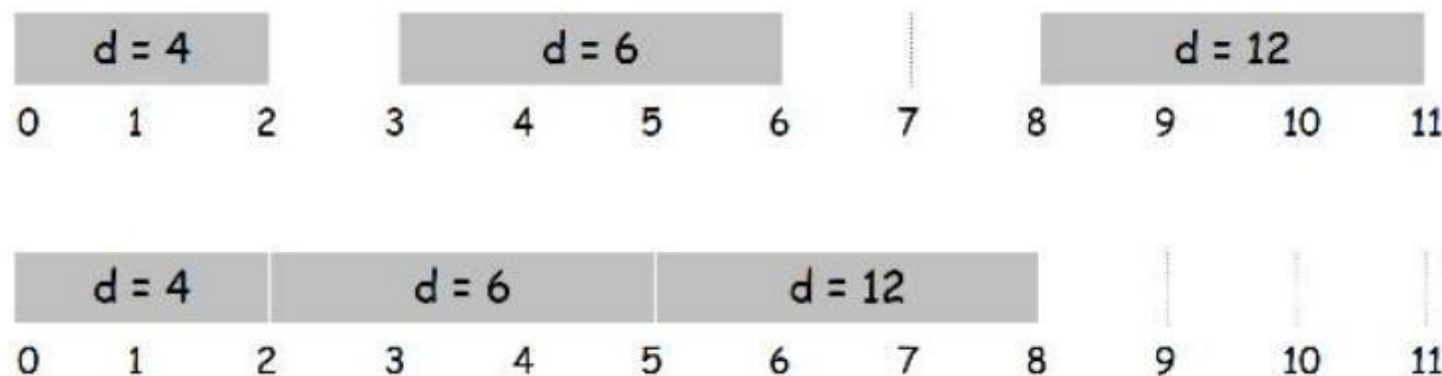
max lateness = 1





Minimizing Lateness: No Idle Time

- **Observation.** There exists an optimal schedule with no **idle time**



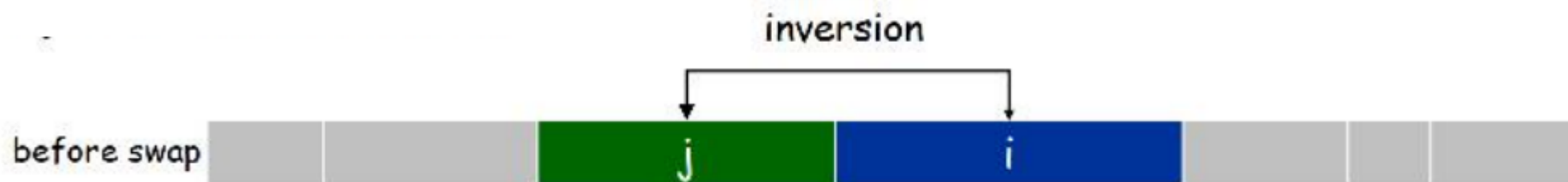
- **Observation.** The greedy schedule has no idle time





Minimizing Lateness: Inversions

- **Def.** An **inversion** in schedule S is a pair of jobs i and j such that: $i < j$ ($d_i \leq d_j$) but j scheduled before i

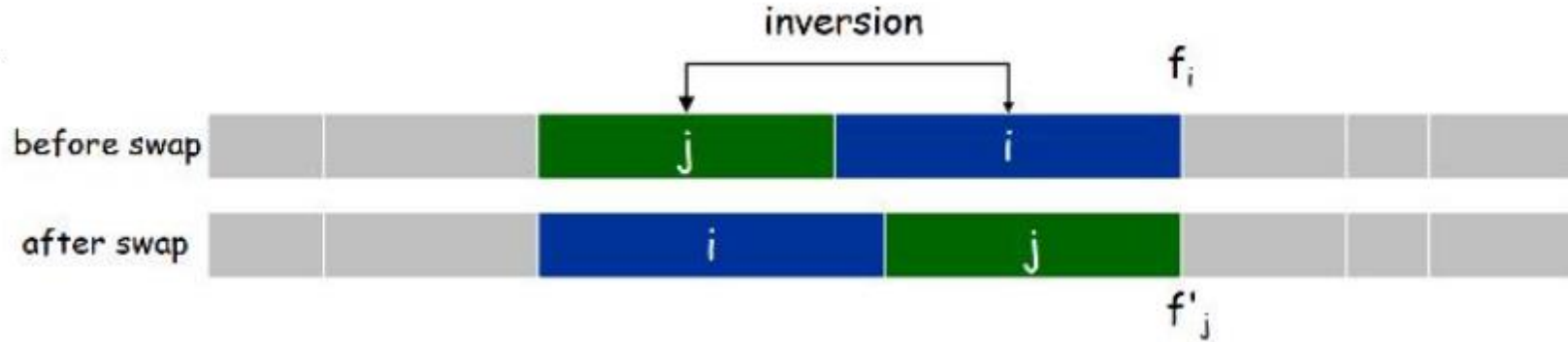


- **Observation.** Greedy schedule has no inversions
- **Observation.** If a schedule (**with no idle time**) has an inversion, it has one with a pair of inverted jobs scheduled consecutively
 - ... j ... i ..., but we know $d_i < d_j$
 - j x_1 x_2 x_3 ... x_m i
 - If we don't have consecutive inversions, then
 - $d_j \leq d_{x_1} \leq d_{x_2} \leq \dots \leq d_{x_m} \leq d_i$
 - $d_j \leq \dots \leq d_i$ but $d_i < d_j$



Minimizing Lateness: Inversions

- Def.** An **inversion** in schedule S is a pair of jobs i and j such that: $i < j$ but j scheduled before i



- Claim.** Swapping two adjacent, inverted jobs reduces the number of inversions by one and does not increase the max lateness

- Pf.** Let I be the lateness before the swap, and let I' be it afterwards

- $I'_k = I_k$ for all $k \neq i, j$

- $I'_i = I_i$

- If job j is late:

$$\begin{aligned}
 \ell'_j &= f'_j - d_j && \text{(definition)} \\
 &= f_i - d_j && \text{(j finishes at time } f_i) \\
 &\leq f_i - d_i && (i < j) \\
 &\leq \ell_i && \text{(definition)}
 \end{aligned}$$

Before swap:

Job j finishes at:

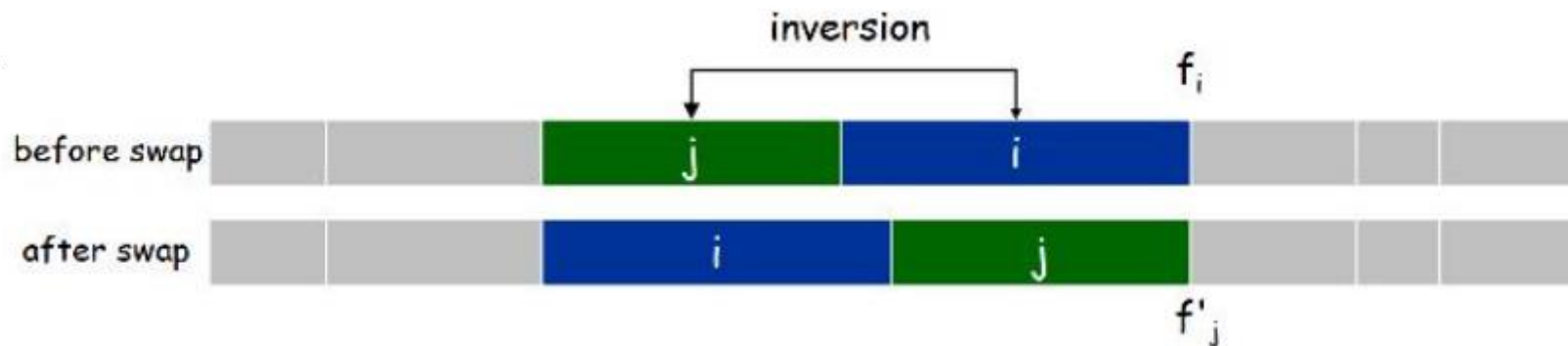
$$f_j = s + t_j$$

Job i starts immediately after j and finishes at:

$$f_i = s + t_j + t_i$$

Minimizing Lateness: Inversions

- Def.** An **inversion** in schedule S is a pair of jobs i and j such that: $i < j$ but j scheduled before i



- Claim.** Swapping two adjacent, inverted jobs reduces the number of inversions by one and does not increase the max lateness

- Pf.** Let I be the lateness before the swap, and let I' be it afterwards

- $I'_k = I_k$ for all $k \neq i, j$

- $I'_i \leq I_i$

- If job j is late:

$$\begin{aligned}
 \ell'_j &= f'_j - d_j && \text{(definition)} \\
 &= f_i - d_j && \text{(j finishes at time } f_i) \\
 &\leq f_i - d_i && (i < j) \\
 &\leq \ell_i && \text{(definition)}
 \end{aligned}$$

$$\ell'_i = f'_i - d_i \leq f_i - d_i = \ell_i$$

After swap:

Job i starts at time s and finishes at:

$$f'_i = s + t_i$$

Job j starts after i and finishes at:

$$f'_j = s + t_i + t_j$$

$$f'_j = s + t_i + t_j = s + t_j + t_i = f_i$$



Minimizing Lateness: Analysis of Greedy Algorithm



- **Theorem.** Greedy schedule S is optimal
- **Pf.** Define S^* to be an optimal schedule that has the fewest number of inversions, and let's see what happens
 - Can assume S^* has no idle time
 - If S^* has no inversions, then $S = S^*$
 - If S^* has an inversion, let $i - j$ be an adjacent inversion
 - Swapping i and j does not increase the maximum lateness and strictly decreases the number of inversions
 - This contradicts definition of S^*





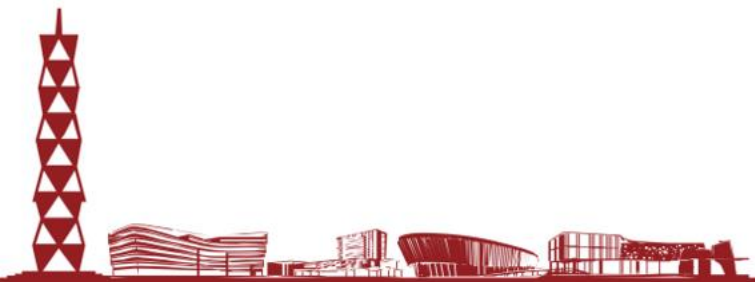
Greedy Analysis Strategies

- **Greedy algorithm stays ahead.** Show that after each step of the greedy algorithm, its solution is at least as good as any other algorithm's
- **Exchange argument.** Gradually transform any solution to the one found by the greedy algorithm without hurting its quality
- **Structural.** Discover a simple "structural" bound asserting that every possible solution must have a certain value. Then show that your algorithm always achieves this bound.





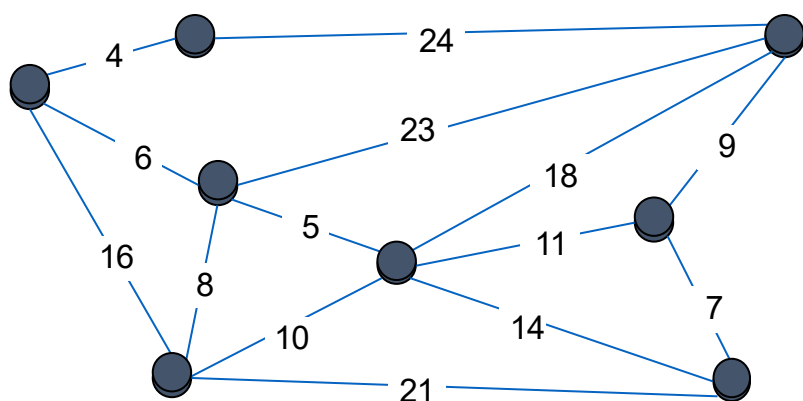
Minimum Spanning Tree



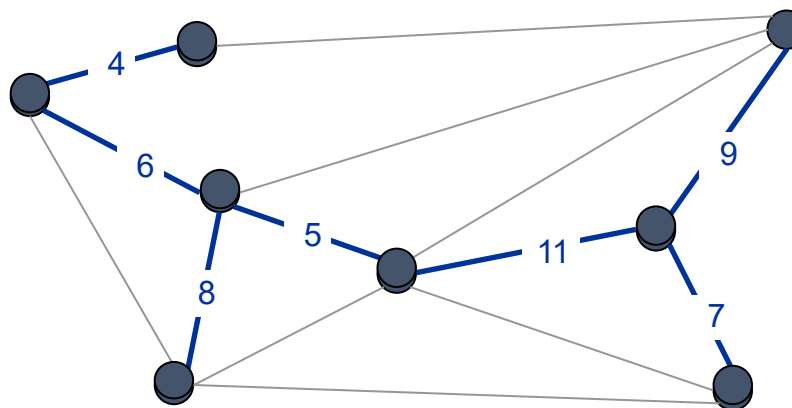


Minimum Spanning Tree

Minimum spanning tree. Given a connected graph $G = (V, E)$ with real-valued edge weights c_e , an MST is a subset of the edges $T \subseteq E$ such that T is a spanning tree whose sum of edge weights is minimized.



$G = (V, E)$



$T, \sum_{e \in T} c_e = 50$

Cayley's Theorem. For a complete graph, there are n^{n-2} spanning trees of K_n .

↑
can't solve by brute force





Applications



MST is fundamental problem with diverse applications.

- Network design.
 - telephone, electrical, hydraulic, TV cable, computer, road
- Approximation algorithms for NP-hard problems.
 - traveling salesperson problem, Steiner tree
- Indirect applications.
 - max bottleneck paths
 - LDPC codes for error correction
 - image registration with Renyi entropy
 - learning salient features for real-time face verification
 - reducing data storage in sequencing amino acids in a protein
 - model locality of particle interactions in turbulent fluid flows
 - autoconfig protocol for Ethernet bridging to avoid cycles in a network
- Cluster analysis.





Greedy Algorithms

Kruskal's algorithm. Start with $T = \phi$. Consider edges in ascending order of cost. Insert edge e in T unless doing so would create a cycle.

Reverse-Delete algorithm. Start with $T = E$. Consider edges in descending order of cost. Delete edge e from T unless doing so would disconnect T .

Prim's algorithm. Start with some root node s and greedily grow a tree T from s outward. At each step, add the cheapest edge e to T that has exactly one endpoint in T .

Remark. All three algorithms produce an MST.



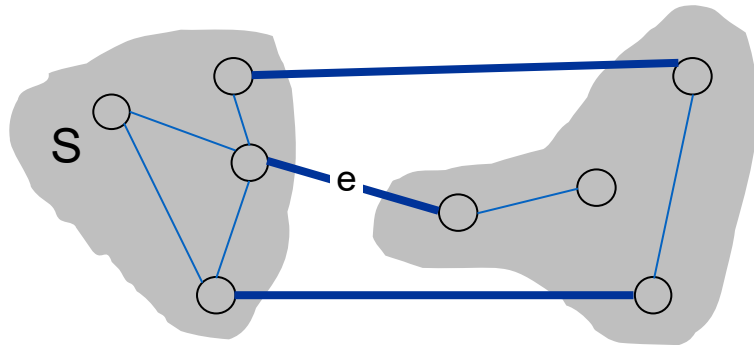


Greedy Algorithms

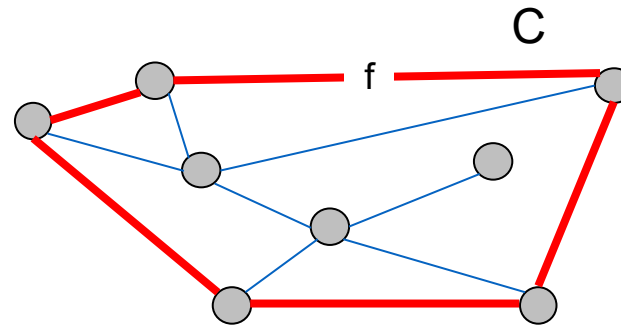
Simplifying assumption. All edge costs c_e are distinct.

Cut property. Let S be any subset of nodes, and let e be the min cost edge with exactly one endpoint in S . Then the MST contains e .

Cycle property. Let C be any cycle, and let f be the max cost edge belonging to C . Then the MST does not contain f .



e is in the MST



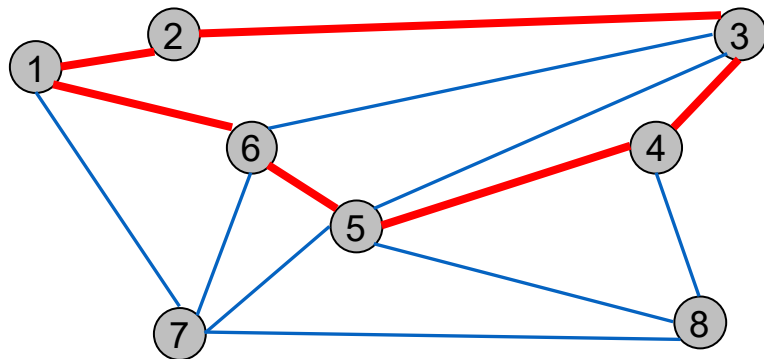
f is not in the MST





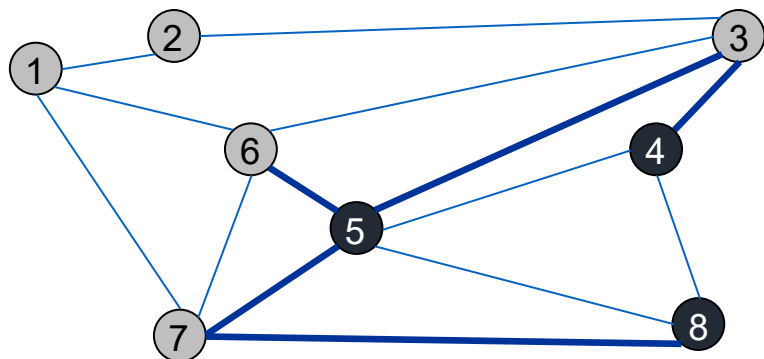
Cycles and Cuts

Cycle. Set of edges the form $a-b, b-c, c-d, \dots, y-z, z-a$.

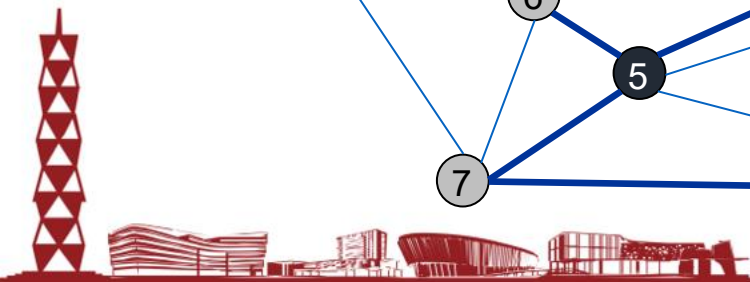


Cycle $C = 1-2, 2-3, 3-4, 4-5, 5-6, 6-1$

Cutset. A cut is a subset of nodes S . The corresponding cutset D is the subset of edges with exactly one endpoint in S .



Cut $S = \{4, 5, 8\}$
Cutset $D = 5-6, 5-7, 3-4, 3-5, 7-8$

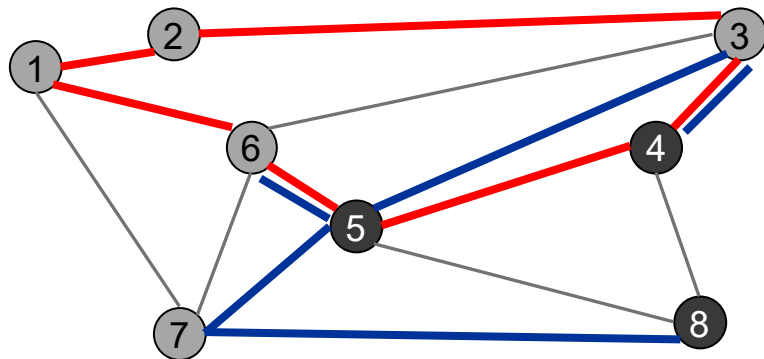




Cycle-Cut Intersection

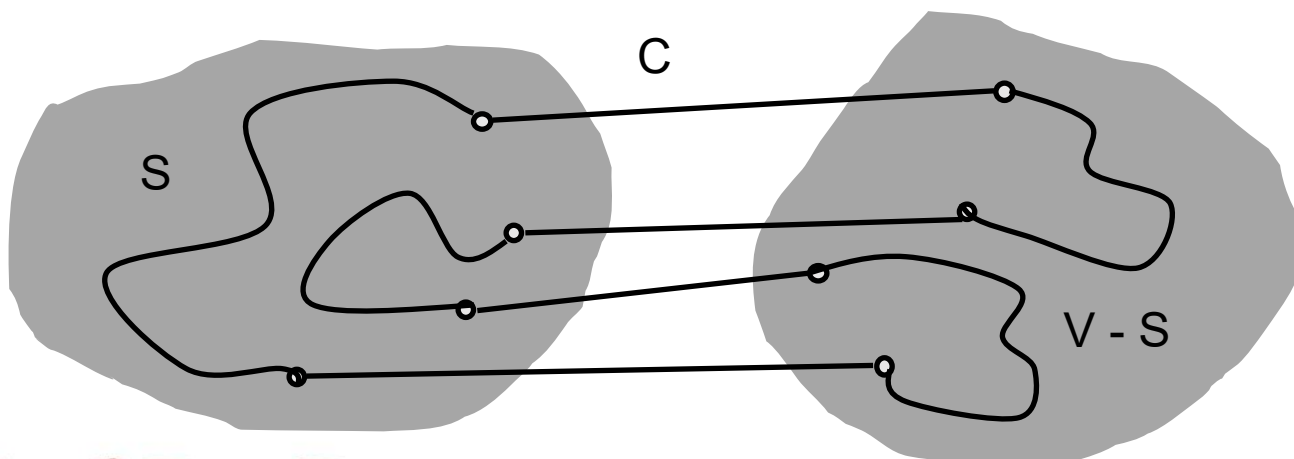


Claim. A cycle and a cutset intersect in an even number of edges.



Cycle $C = 1-2, 2-3, 3-4, 4-5, 5-6, 6-1$
Cutset $D = 3-4, 3-5, 5-6, 5-7, 7-8$
Intersection = $3-4, 5-6$

Pf. (by picture)





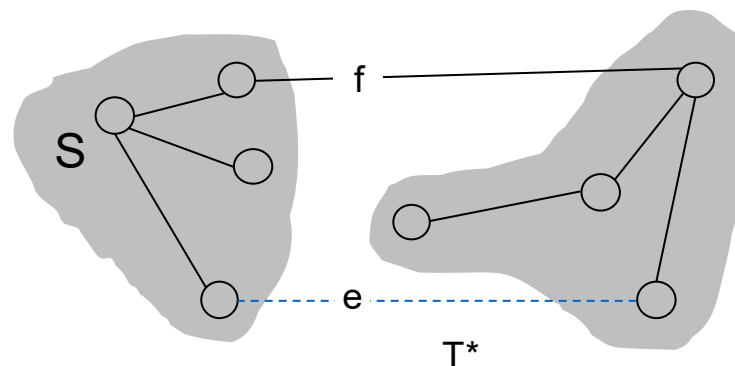
Greedy Algorithms

Simplifying assumption. All edge costs c_e are distinct.

Cut property. Let S be any subset of nodes, and let e be the min cost edge with exactly one endpoint in S . Then the MST T^* contains e .

Pf. (exchange argument)

- Suppose e does not belong to T^* , and let's see what happens.
- Adding e to T^* creates a cycle C in T^* .
- Edge e is both in the cycle C and in the cutset D corresponding to $S \Rightarrow$ there exists another edge, say f , that is in both C and D .
- $T' = T^* \cup \{e\} - \{f\}$ is also a spanning tree.
- Since $c_e < c_f$, $\text{cost}(T') < \text{cost}(T^*)$.
- This is a contradiction. ▀





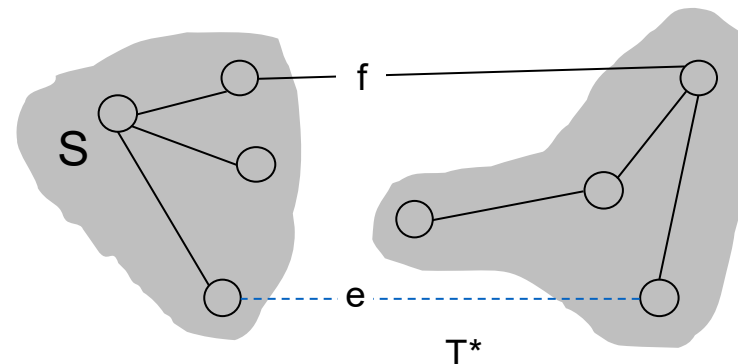
Greedy Algorithms

Simplifying assumption. All edge costs c_e are distinct.

Cycle property. Let C be any cycle in G , and let f be the max cost edge belonging to C . Then the MST T^* does not contain f .

Pf. (exchange argument)

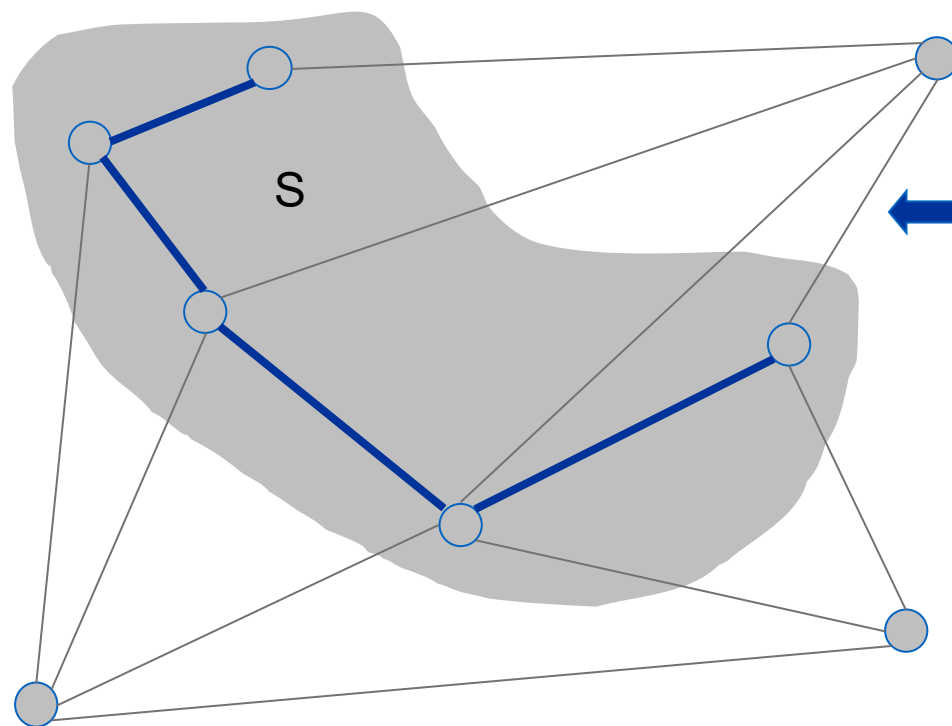
- Suppose f belongs to T^* , and let's see what happens.
- Deleting f from T^* creates a cut S in T^* .
- Edge f is both in the cycle C and in the cutset D corresponding to $S \Rightarrow$ there exists another edge, say e , that is in both C and D .
- $T' = T^* \cup \{e\} - \{f\}$ is also a spanning tree.
- Since $c_e < c_f$, $\text{cost}(T') < \text{cost}(T^*)$.
- This is a contradiction. ▀



Prim's Algorithm: Proof of Correctness

Prim's algorithm. [Jarník 1930, Dijkstra 1957, Prim 1959]

- Initialize S = any node.
- Apply cut property to S .
- Add min cost edge in cutset corresponding to S to T , and add one new explored node u to S .





Implementation: Prim's Algorithm



Implementation. Use a priority queue ala Dijkstra.

- Maintain set of explored nodes S .
- For each unexplored node v , maintain attachment cost $a[v]$ = cost of cheapest edge v to a node in S .
- $O(n^2)$ with an array; $O(m \log n)$ with a binary heap.

```
Prim(G, c) {  
    foreach (v ∈ V) a[v] ← ∞  
    Initialize an empty priority queue Q  
    foreach (v ∈ V) insert v onto Q  
    Initialize set of explored nodes S ← ∅  
  
    while (Q is not empty) {  
        u ← delete min element from Q  
        S ← S ∪ { u }  
        foreach (edge e = (u, v) incident to u)  
            if ((v ∉ S) and (ce < a[v]))  
                decrease priority a[v] to ce  
    }  
}
```

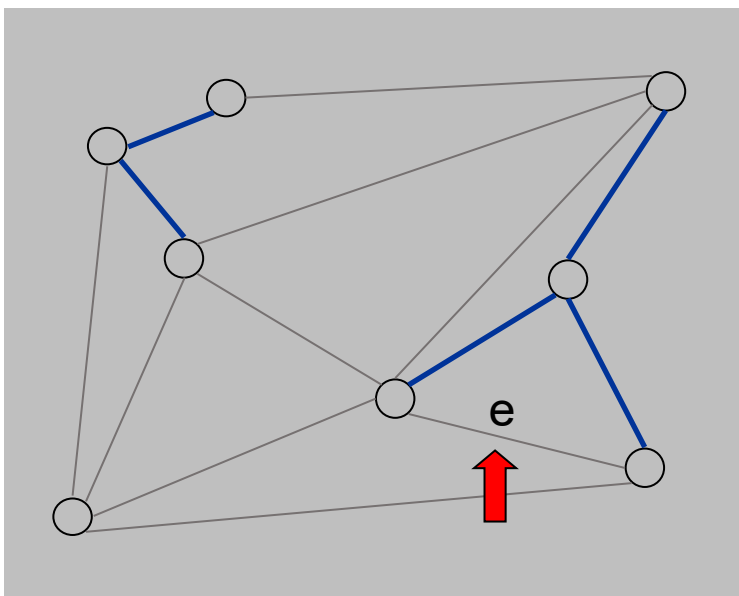




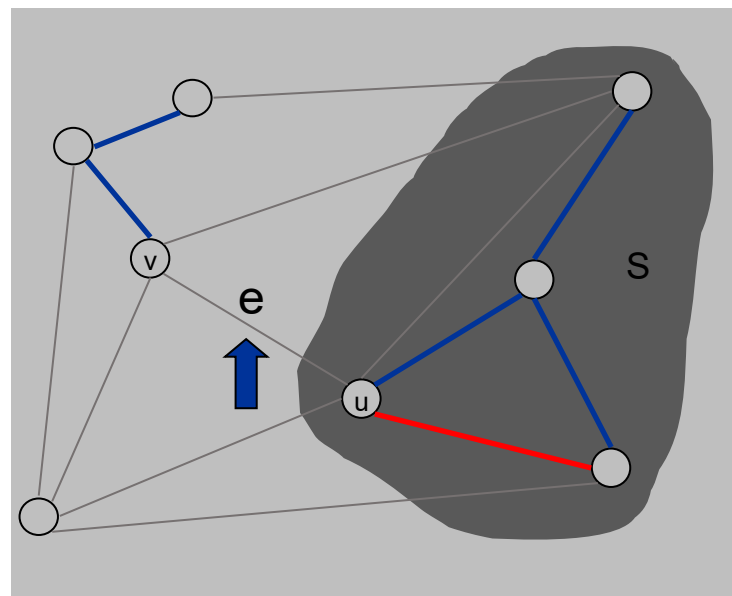
Kruskal's Algorithm: Proof of Correctness

Kruskal's algorithm. [Kruskal, 1956]

- Consider edges in ascending order of weight.
- Case 1: If adding e to T creates a cycle, discard e according to cycle property.
- Case 2: Otherwise, insert $e = (u, v)$ into T according to cut property where S = set of nodes in u 's connected component.



Case 1



Case 2





Implementation: Kruskal's Algorithm

Implementation. Use the **union-find** data structure.

- Build set T of edges in the MST.
- Maintain set for each connected component.
- $O(m \log n)$ for sorting and $O(m \alpha(m, n))$ for union-find.

$m \leq n^2 \Rightarrow \log m$ is $O(\log n)$ essentially a constant

```
Kruskal(G, c) {  
    Sort edges weights so that  $c_1 \leq c_2 \leq \dots \leq c_m$ .  
     $T \leftarrow \emptyset$   
  
    foreach ( $u \in V$ ) make a set containing singleton  $u$   
  
    for  $i = 1$  to  $m$   
        ( $u, v$ ) =  $e_i$       are  $u$  and  $v$  in different connected components?  
        if ( $u$  and  $v$  are in different sets) {  
             $T \leftarrow T \cup \{e_i\}$   
            merge the sets containing  $u$  and  $v$   
        }  
    return  $T$       merge two components  
}
```





Lexicographic Tiebreaking

To remove the assumption that all edge costs are distinct: perturb all edge costs by tiny amounts to break any ties.

Impact. Kruskal and Prim only interact with costs via pairwise comparisons. If perturbations are sufficiently small, MST with perturbed costs is MST with original costs.

↑
e.g., if all edge costs are integers,
perturbing cost of edge e_i by i / n^2

Implementation. Can handle arbitrarily small perturbations implicitly by breaking ties lexicographically, according to index.

```
boolean less(i, j) {  
    if      (cost(ei) < cost(ej)) return true  
    else if (cost(ei) > cost(ej)) return false  
    else if (i < j)                 return true  
    else                           return false  
}
```





Next Time: Greedy Algorithms (Cont.) & Divide and Conquer

